

## Abstract

This thesis applies the recent trends in parallel processing using a Graphics Processing Unit (GPU) to the field of Molecular Communications (MC), an investigation into communication possibilities of futuristic *in vivo* nanomachines. Existing MC simulations do not yet account for structural boundaries, various obstructions (e.g. red blood cells), or the ability to simulate a massive number of interactive MC messenger molecules. The messenger molecules are subject to Brownian motion and a directional laminar flow, before any outside interaction. This means that a number of molecules released from the same starting location are highly unlikely to be found at the same location some time later. By utilizing a GPU, these paths can be calculated on a massive scale. The end points can then be interpreted using stochastic processes to better estimate the probable end location of a molecule, given its starting point. The use of a GPU also allows for other obstacles and complications to be added to the path of those molecules. This tool should enable and expedite existing as well as future study in the MC field.

## **Acknowledgement**

A special thank you to Dr. Yun Tian and Dr. Uri Rogers. You have both been an incredible help through this process. Your guidance and extensive knowledge of your respective fields has been invaluable through out this project. And a thank you to Dr. Andrew Oster for acting as my third committee member.

## Contents

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                           | <b>1</b>  |
| <b>2</b> | <b>Related Works</b>                          | <b>2</b>  |
| 2.1      | Molecular Communication . . . . .             | 2         |
| 2.2      | Graphics Processing Unit Background . . . . . | 4         |
| <b>3</b> | <b>Methodology</b>                            | <b>7</b>  |
| 3.1      | Molecule Simulation . . . . .                 | 7         |
| 3.2      | Data storage . . . . .                        | 10        |
| 3.3      | Options . . . . .                             | 11        |
| <b>4</b> | <b>Evaluation</b>                             | <b>11</b> |
| 4.1      | Validation . . . . .                          | 12        |
| 4.2      | Speed Comparison . . . . .                    | 14        |
| <b>5</b> | <b>Discussion</b>                             | <b>17</b> |
| <b>6</b> | <b>Future Work</b>                            | <b>18</b> |
| <b>7</b> | <b>Conclusion</b>                             | <b>19</b> |

# 1 Introduction

Nanotechnology is a new and emerging field, and has been studied in recent years as new ways of interacting with the body, *in vivo* [1]. Nanotechnologies have the possibility of delivering drugs to exact areas in the body, acting as *in vivo* imaging probes or providing early stage disease detection [2]. However, many of the capabilities hinge on the existence of a suitable network. The networks currently available do not meet the size requirement needed to exist within the human body. Existing networks generally fall within one of two classifications, wired or wireless. Wired technologies use a size-able amount of power as well as presenting an installation problem, while wireless technologies require antenna sizes that are generally too large to implement *in vivo* [2]. In order to occur at a size applicable to both geometric and resource constraints, one can look to nature for a suitable, idealistic communication method. Molecular communication focuses on utilizing molecules in the blood to communicate, similar in form to the communication patterns platelets [3]. Passive and active transportation techniques are being studied, but for this study passive transport is examined [1, 2]. As the molecules are transported through the body, the flow and contents of the blood affect the position and path of those particles. Red blood cells have a significant effect on the blood stream due to their relative size to other particles [4]. cursory research models explore molecular movement; however, they are not suitable for statistical analysis needed for molecular communication. In a previous paper we have shown that with multiple receivers, non-linear spatial dependency can be found[5]. These findings emphasize the challenge in dealing with particles in the blood stream and the inability to predict where the particles will be or how likely they are to make it to a specific point. With radio communications packets with multiple bits of data can be sent and received, with molecular communication, this is not necessarily possible. The particles will more likely represent a binary value. Thus, the need to be able to control and understand closely the sending and retrieval of these packets. Each of molecules has a distinct path, meaning that even if multiple molecules were released from the same point at the same time, the likelihood of any of those molecules reaching the same destination is highly improbable.

GPUs have been used for massively parallel processes in many fields, originally beginning with computer graphics. They are excellent at processing massive number of similar commands, they are not ideal for general purpose computing. Due to the parallel nature of the problem, though not so-

called, "embarrassingly parallel, GPUs are a good fit for the application of this project. MC requires an understanding of how molecules will move in the blood stream, though since each molecule will have a different path a stochastic model is needed. In order for the data in this model to be as accurate as possible very large simulations of molecules are needed. This can be enabled by the application of a GPU. In a study of the location of a molecule, each simulated molecule is independent of each other molecule. This allows each molecular time step or reflection to be calculated independent of each other molecule, best utilizing the parallel ability of the GPU. By calculating one step at a time thousands of molecular paths can be calculated in parallel. This paper discusses the simulation of these particles using a GPU.

The paper is organized in the following manner. Section 2 will explore existing studies in molecular communication and look at ways GPUs have been implemented in other fields. Section 3 will discuss the implementation of molecular simulation on the GPU and design decisions. Section 4 will analyze the results achieved and verify their accuracy, while also looking at the time-cost improvements and added functionality. Section 5 will be a discussion of the findings noted in 4, taking a closer look at the newly available data. Section 6 will discuss future work and other possible applications. Section 7 will contain closing remarks and observations.

## **2 Related Works**

This paper focuses on the application of GPUs to the simulation of particle movement for use in the molecular communication field. This section will first discuss the background of molecular communication and its application, then delve into the history and applied uses of the GPU.

### **2.1 Molecular Communication**

The idea of nano-technologies in the body have long existing in the stories of science fiction. Recently, these dreams and stories have been approaching reality. Nano-devices currently exist in medicine, however in most of these implementations, the devices function independently from each other in the bloodstream. While currently available nanomachines can do things such as, recover from glucose feedback loops using nanopores, the complexity of these devices is limited due to power, size and communication restrictions [2]. To expand the functionality of devices in the blood stream many communication methods have been explored, such as molecular, nano-mechanical,

electromagnetic and acoustic communications [2]. Among these, molecular and acoustic communication are most bio-compatible. This paper will focus on molecular communication due to its similarity with natural bodily processes and the current availability of molecular communication equipment [2].

Molecular communication uses naturally existing molecules to communicate messages between two things. This is seen naturally in the way white blood cells (WBC) communicate. For example, WBC will release special proteins called cytokines that can signal to other cells to change their behavior to assist in fighting an infection. There are several different ways *in vivo* molecular communication has been approached, but most can be classified as either a passive or active transport. Active transport can use routes such as a molecular rail or microtubules [6]. While this in theory enables very consistent communication, the implementation and scalability remain a challenge. Passive transport relies on the use of outside forces to connect the two media. Within the body there are two main forces, among many others, that act upon a molecule, Brownian motion and blood flow. Brownian motion is known to act on molecules in a fluid. This perceived movement is caused by the collision of other particles with these molecules. Blood flow can be divided up into two different areas when viewed from a high level, a core region where most cells are found and a cell free layer[4]. A turbulent flow in the core region is caused by the tumbling of Red Blood Cells (RBCs), which are magnitudes larger than most other articles in the blood. In Tan et al, a model for the tracking and influence of RBC on nanoparticles (NP) is discussed using mean square displacement [4]. The turbulent flow pushes the smaller particles to the edges of the blood vessel, called the cell free layer, where they travel in a smoother flow referred to as laminar flow. For this paper, RBCs were considered out of scope, but are important and should be looked at in future works. Laminar flow provides an ideal stream for passive MC. The movement of a given particles in the cell free layer is, thus affected by Brownian motion and laminar flow.

In order to model the individual molecular paths, it is important to understand how a molecule behaves. The Brownian motion, can be simulated using the forward Kolmogorov//Fokker-Planck equation

$$\frac{\partial p_B(t, \xi; \xi_0, t_0)}{\partial t} = \frac{1}{2} \sum_{j=1}^3 \sum_{k=1}^3 \frac{\partial^2 D_{j,k} p_B}{\partial \xi_j \partial \xi_k} - \sum_{j=1}^3 \frac{\partial v_j p_B}{\partial \xi_j} \quad (2.1)$$

where  $D_j, k$  is a diffusion constant,  $v_j$  is the drift velocity in direction  $j$  and  $p_B(t, xi; t_0, xi_0)$  is a pdf for emission location  $xi$  at time  $t$  with starting points of  $xi_0$  and  $t_0$  respectively [5]. The first portion of the equation can be thought of as the random motion with the second part of the equation accounting for the drift. For this paper, the laminar flow is assumed to be in a singular direction, thus the parameter is only applied to the  $j$  direction. The velocity is representative of the speed of the flow, and the  $\Delta T$  is precision of the time step used due to the lack of a digital continuous option.

## 2.2 Graphics Processing Unit Background

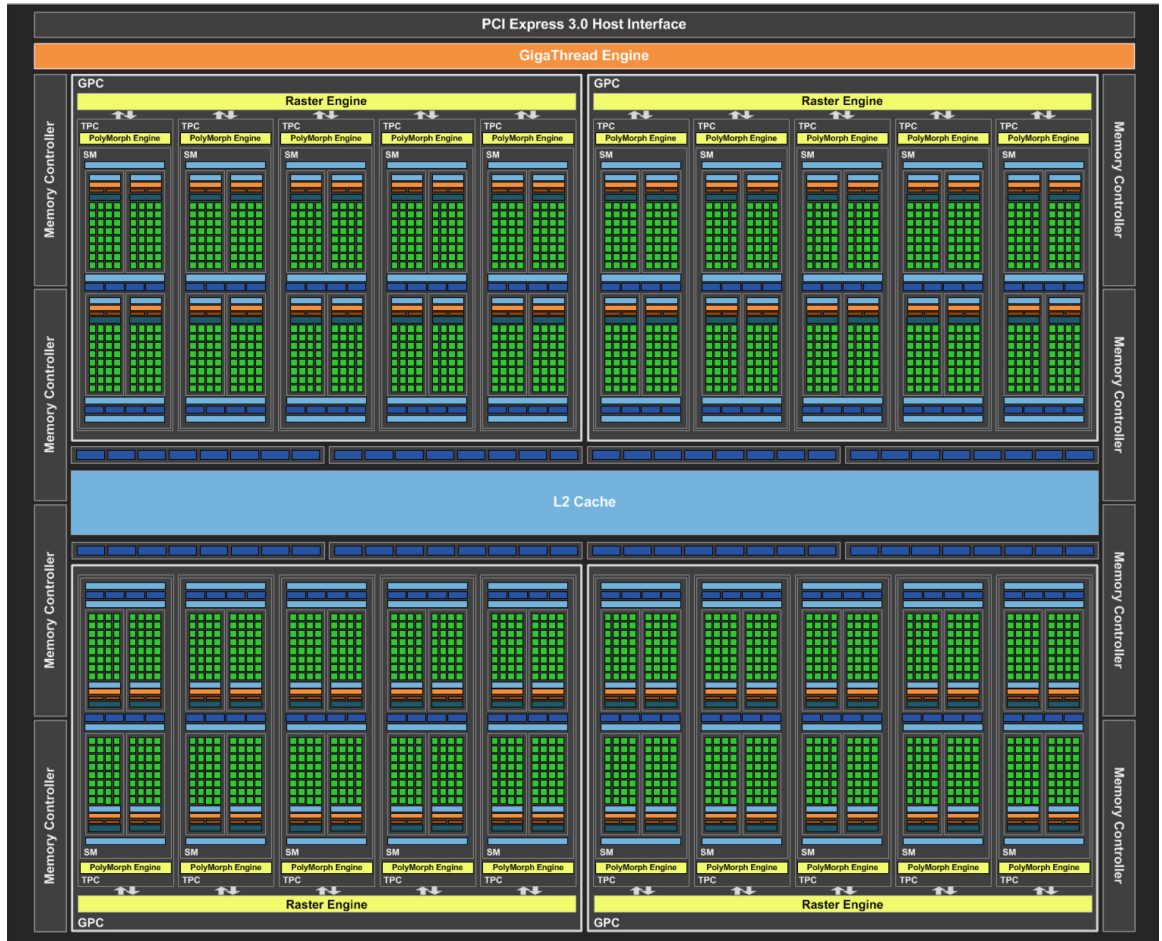


Figure 2.1: Block Diagram of the GP104 GPU [7]

Today the GPU is applied to many unique fields in order to provide speed ups for inherently parallel problems, but originally the GPU was mainly focused on graphics generation. This trend continued until around 2007, when Nvidia announced a beta version of CUDA, a C based programming language, that could be used to harness the parallel nature of its video cards. This alternative use of the

GPU became known as general purpose GPU usage, or GPGPU [8]. While the new technological advancement was exciting, GPUs work differently than the tradition CPU, so adjustments needed to be made. While the CPU is focused on general purpose task needed for every day use, the GPU, 2.1, is made up of many streaming microprocessors (SMs), 2.2. These SMs are highly parallel multiprocessors that schedule groups made of 32 threads, called warps, to the CUDA cores [7]. Each SM contains some number of CUDA or scalar cores, in the case of 2.2, 128 CUDA cores are on each SM. It is not uncommon for consumer CPUs to have 2-4 cores, compare this to the number in the Nvidia GeForce GTX 1070, a mid-range consumer GPU, at 1920 CUDA cores. It then makes sense that CPUs focus on minimizing latency while GPUs focus on maximizing throughput. CUDA programs are based around Kernels, which are function calls, highly parallel in nature, made of thousands of threads. These threads are grouped into blocks and ultimately grids that the Kernel must control. Each of these threads run an identical function, independent of each other thread in both result and completion order. Memory exists on both the Host (CPU) and Device (GPU) side and must be transferred back and forth from host and device in ordered to be accessed by the respective sides. This means that data must be transferred to device memory in order to be acted upon by the GPU. Subsequently, it must be transferred back to host memory in order to be referenced by the CPU. Thus, for highly parallelizable or so-called embarrassingly parallel problems the GPU is commonly superior to the CPU in processing time required. These speed inequalities only exacerbated as the number of identical tasks increase. Though for a small number of parallel tasks, the CPU is generally faster. This is partially due to the required memory transfer between the host and device. In the paper GPU Computing, Owens *et al.*, shows that for a task of scanning elements, a CPU is faster at scanning elements when there are  $10^4$  or fewer, but is quickly surpassed as the number of elements increases[8]. The GPU has been used in many different fields including applications in Stochastic Differential equations [9].



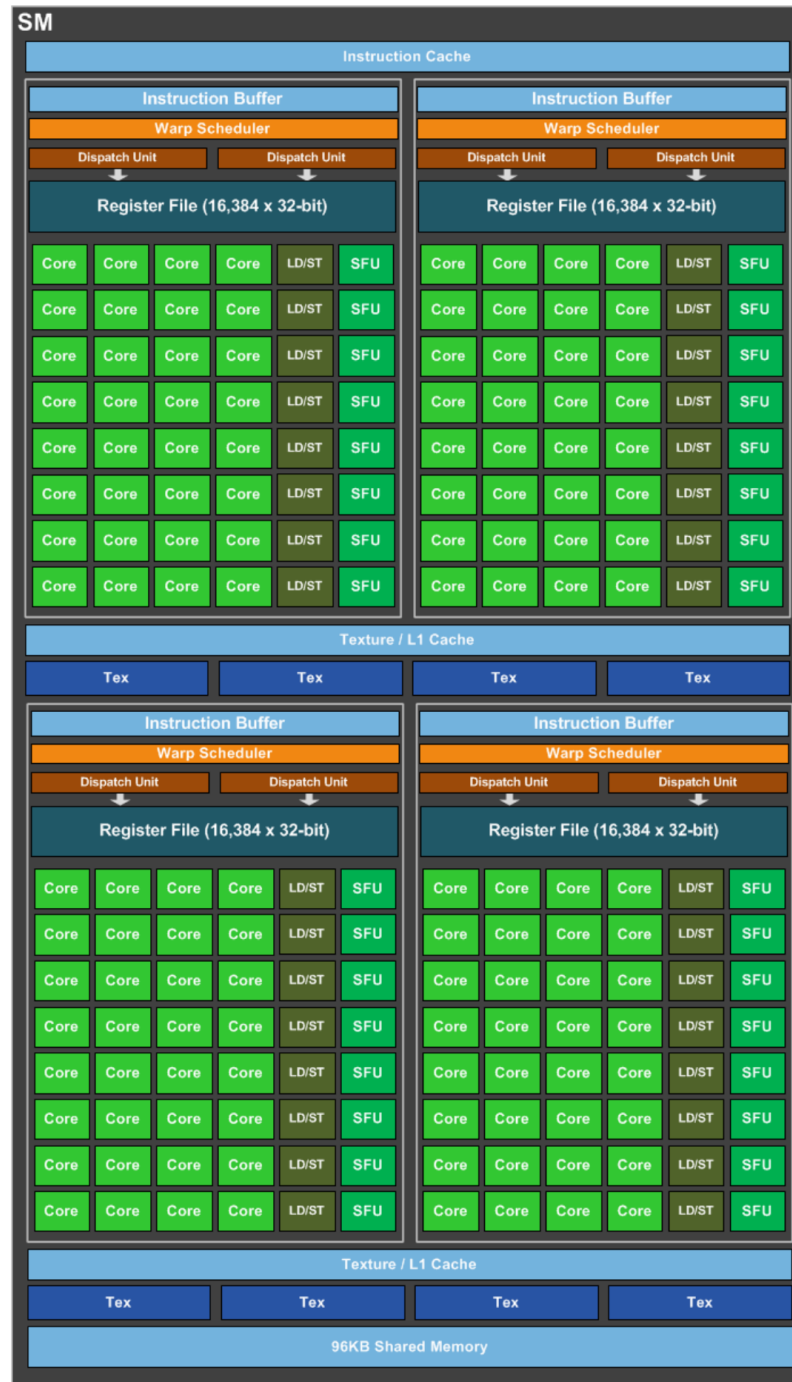


Figure 2.2: GP104 Streaming Multiprocessor Diagram [7]

### 3 Methodology

#### 3.1 Molecule Simulation

The starting point for this project looks at what the movement of a particle would look like. This has been discussed in our previous paper[5]. Particle movement is based off of a Brownian motion and can be represented as stated in 2.1. Laminar drift can be assumed, by either ignoring RBCs in the short term, or assuming particle movement is limited to the cell free layer of blood vessels. The tumbling motion of the RBC in the blood stream which forces most smaller molecules, such as the proposed nanomachine into this layer. Assuming, distributed detection, a passive form of molecular communication is used, the released molecules will need to be detected downstream in order to communicate. This means the molecules released up stream will have one of two possible routes from the view of the receiver. Either the molecule will reach the receiver, or it will miss. If a molecule takes longer than a set amount of time the molecule is also considered to have missed. This simulation has been performed in, Dr. Rogers paper, and was used as a baseline simulation [1, 10]. For this simulation, a singular starting point at some  $\xi_0(x_0, y_0, z_0)$ , and a static receiver are assumed. This means that each molecule will behave according to a Brownian motion, and will be tracked for a chosen period of time,  $t$ . If at any point that molecule enters the receiver, the first-hit time is recorded, no subsequent times are recorded. If by time,  $t$ , the molecule has not been detected, then it is assumed to have missed. With a static diffusion rate, the output of the simulation can be compared and validated. The solutions have been implemented on both the CPU and GPU for comparison purposes. In order to implement the algorithm was adapted to,

$$\sqrt{2 * Db * \Delta t} * randn,$$

where  $Db$  is the diffusion factor,  $v_j$  is the velocity in the  $j$  direction, the  $\Delta T$  is the time step size and  $randn$  represents a random normalized value. This equation is used for the  $x$ ,  $y$  and  $z$  directions, when Laminar drift is assumed absent. This equation represents the path of a single molecule to an end time,  $t_{end}$ . To gain a stochastic understanding of the probability of this single particle hitting the receiver, this simulation must be repeated a significant number of times. The CPU code is written in C++, and the GPU code is C++/CUDA. The CPU was implemented using an  $\Theta(N^2)$  algorithm,

$x_0 = 0; y_0 = 0; z_0 = 0;$

**For**  $P < \text{numberofpaths}$  **do**

**For**  $Ts < \text{numberoftimesteps}$  **do**

$x_{next+} = \sqrt{2 * Db * \Delta t * rand_x};$

$y_{next+} = \sqrt{2 * Db * \Delta t * rand_y};$

$z_{next+} = \sqrt{2 * Db * \Delta t * rand_z};$

: CPU implementation of Brownian Motion

A random number generator (RNG) is required for this equation. The built-in RNG was not suitable for our needs due to the lack of a Normal distribution. In order to get the RNG needed the Marsaglia polar method was used [11]. The Normalized random number generator was determined valid using cross correlation in comparison to MATLABs randn output. It can be seen that the peaks nearest the center peak are small in size showing that little correlation is found?

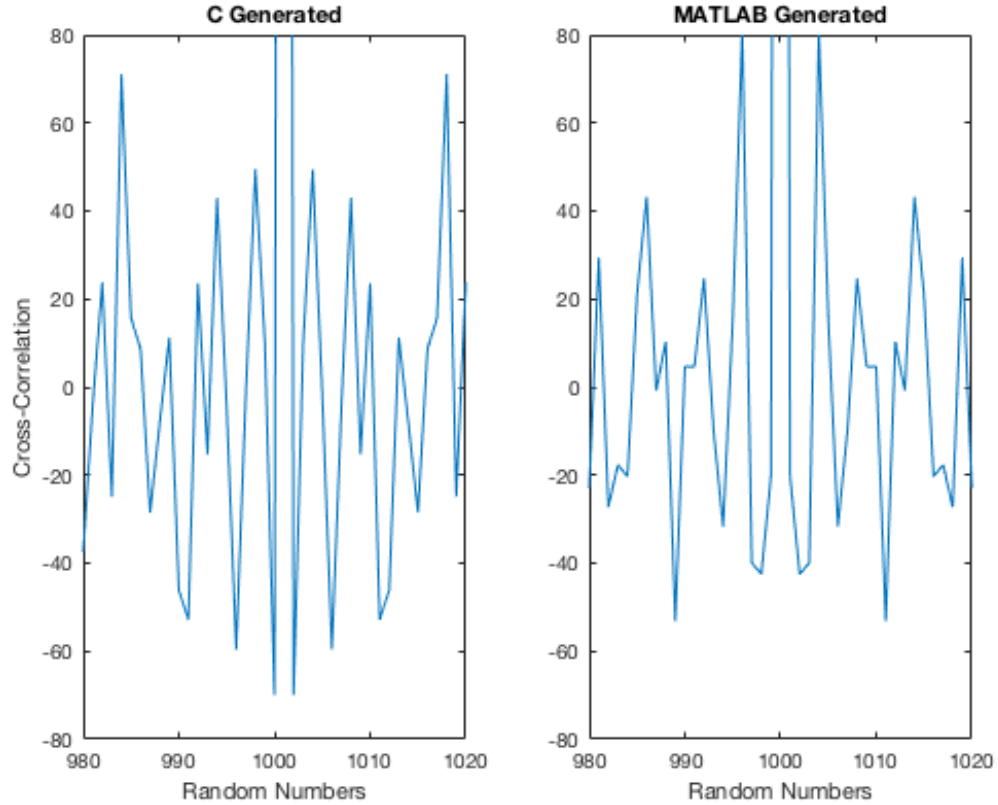


Figure 3.1: The Cross-Correlation of C function compared with MATLAB.

This route, was not needed for the GPU implementation as a built-in function existed and could

be used. The same cross correlation validation was used to determine the validity of the existing function and it was found to be acceptable.

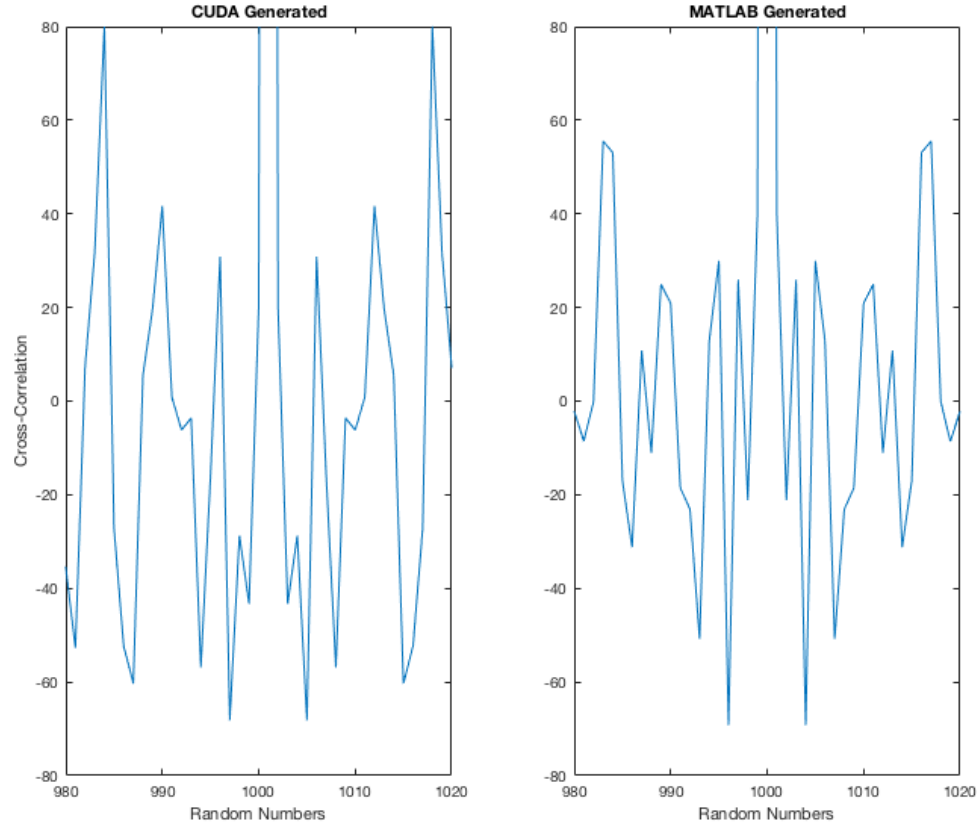


Figure 3.2: The Cross-Correlation of CUDA function compared with MATLAB.

Moving the equation to a parallel structure required a different approach. While the CPU is traditionally based on a Single instruction stream single data stream (SISD) structure, the GPU is best utilized when using Single instruction stream, multiple data streams (SIMD) structure. For the molecular movement via Brownian motion, each step is dependent on the last in the form

$$x_{new} = x_{last} + formula,$$

this was not an ideal fit for the GPU. The GPU is able to calculate a large number of items in parallel, however they are not guaranteed to be started or completed in any given order. This means that if the same implementation was used, the movement of the molecule would not be guaranteed to move in

time order.  $t_0$  could come after  $t_n$ , where  $n \geq 0$ . Thus, instead of calculating a  $N$  number of paths at a time,  $N$  paths are calculated one time step at a time. A path can be defined as the route a molecule takes in a given time period,  $t$ . An example path can be seen in 4.3. This means that if 1000 paths were to be calculated, for each path the position value at  $t_0$  would be found. Once all calculations for  $t_0$  had been calculated,  $t_1$  is then calculated in the same way, continuing this pattern to  $t_n$ . By approaching the calculation in this manner, the output can be passed forward each time without any reliance on past data, whilst still taking advantage of the parallel structure needed required for the GPU. This inherently implies that the GPU algorithm will be better at calculating a large number of paths than it will paths that are followed for a long period of time. This differs from the CPU implementation as the CPU calculates an entire path to  $t_{end}$ , then moves on to the next path. Thus, each path is calculated to its entirety, one after another on the CPU.

```

local  $idx \leftarrow blockIdx.x \times blockDim.x + threadIdx.x$ 
load  $iter, velocity, D_b, x, y$ , and  $z$  parameters and store as local variables
Set  $rng$  seed and use to create  $rand_x, rand_y, rand_z$ 
If  $idx < iter$  then
   $x_{next} += \sqrt{2 * D_b * \Delta t * rand_x};$ 
   $y_{next} += \sqrt{2 * D_b * \Delta t * rand_y};$ 
   $z_{next} += \sqrt{2 * D_b * \Delta t * rand_z} + velocity \times \Delta t;$ 

```

Algorithm 1: A Cuda kernel to calculate Brownian motion with Laminar drift

The next thing to be added to the simulation was the effect of laminar flow (aka Brownian drift). The force is assumed to be applied in a singular  $z$  direction. By applying a velocity in the direction of  $z$  at each time step we can simulate the drift. This makes the full equation for the  $z$  direction,

$$z_{next} += \sqrt{2 * D_b * \Delta t * rand_z} + vel * \Delta t.$$

### 3.2 Data storage

Next, a way of storing the data needed to be established. The structure needed to be efficient or simple due to the number of iterations it would be expected to work with. It also needed to be able to quickly access points in a similar area. The first route looked at was a tree-structure of some type. Octrees and Kd-trees were both looked at, and carefully considered early in the project. However, once it was decided that the data would be calculated in steps, it made sense to instead do real-time calculations when a collision was detected. Due to the known three-dimensional position

of all molecules and obstacles at a time,  $t_n$ , the reflections of those particles could easily be added into the next time step. As the number of collisions increase it may still make sense to use a tree structure. A tree structure may also make sense if the data needs to be acted on after leaving the GPU.

### 3.3 Options

The program has a variety of available outputs. It can output, every three-dimensional point for each time step allowing the graphing of a full molecular path. This is great for troubleshooting, as well as understanding what the paths will typically look like. However, as the number of paths increase, the data becomes massive, and more difficult to use. The data quickly grows files of 8GB and only increases from there. In order to reduce this issue, while still collecting important data, the option for a hit-time was added. This means that only the particles that make it to the set obstacle are recorded, be it a 1D plane or a receiver. At each time step the particle will be compared to the hit-area and recorded if it is found to be inside. This means that for each particle that achieves the set condition it is possible, if not likely, to have several points recorded. A similar option, records only the first-hit time. This means that every time after the first hit and all non-hitting times are discarded. These concessions allow for significantly larger data sets to be recorded. To implement the collisions, the bounds for the specific objects are set with in the code. For things such as the walls of the blood vessel, some parameters, in this case the radius, are made available for the user to set. Collisions have been designed for the hit-receiver, a planar limit and the blood vessel walls. By implementing the collisions in this manner, the GPU can be utilized to perform these checks and reactions in real-time, instead of having to store and then re-access the data. Other obstacles maybe added in the future and will be discussed in Section 6.

## 4 Evaluation

A goal of this thesis was to enable study that had be difficult or unavailable at the time this paper was written. In this pursuit, two main items had to examined, the speed of the analysis and the accuracy of the findings. The speed at which the simulation operates is important in order to be able to quickly run different experiments and compare the results. The time in which a test can be completed, has an effect of the number of tests as well as the willingness to run a test. Due to the

stochastic nature of data produced, it is very important to have a large sample size.

#### 4.1 Validation

In order to compare the speed of the processes it was important to first identify some bench marks that could be compared on both the CPU and the GPU. It was also vital that the algorithms used could be compared to and validated by previously vetted results. From this perspective, two main tests were implemented.

The first test implemented looks at the paths of the particles in 3-dimensional space. Given a starting point,  $\xi_0$ , in  $\mathbb{R}^3$  the test records the time each path takes to reach a receiver of set radius and a given  $\xi_{end}$ . The receiver has a radius,  $a$ , and exists a distance,  $d$  from  $\xi_0$ . For each particle that reaches the receiver, the first-hit time is recorded. For this simulation, no walls, drift or other variables are considered. The particles move in free space according to Brownian motion and those that reach the receiver are recorded.

If the detecting surface is supposed to be perfectly absorbing, then the equation for the so-called reaction rate at  $r = a$  for a receiver of radius  $a$ , can be written

$$H_{Diff}(t|d, t_0) = \frac{a}{d} \frac{1}{\sqrt{4\pi D(t-t_0)}} \frac{d-a}{t-t_0} \times \exp \left[ -\frac{(d-a)^2}{4D(t-t_0)} \right], \quad (4.1)$$

where  $t$  is current time step,  $t_0$  is the start time, and  $d$  is the distance of release point to the receiver [1]. The hit probability for a pure diffusion process can be then be calculated using,

$$Pr(m_k hit \mathbf{R}) = \frac{a}{d}. \quad (4.2)$$

This equation is justified by Schulten *etal.* by massaging a well known diffusion equation, then taking the limit as  $(t - t_0) \rightarrow \infty$ , with the assumption of a perfect absorbing surface[12, Sec,3.4].

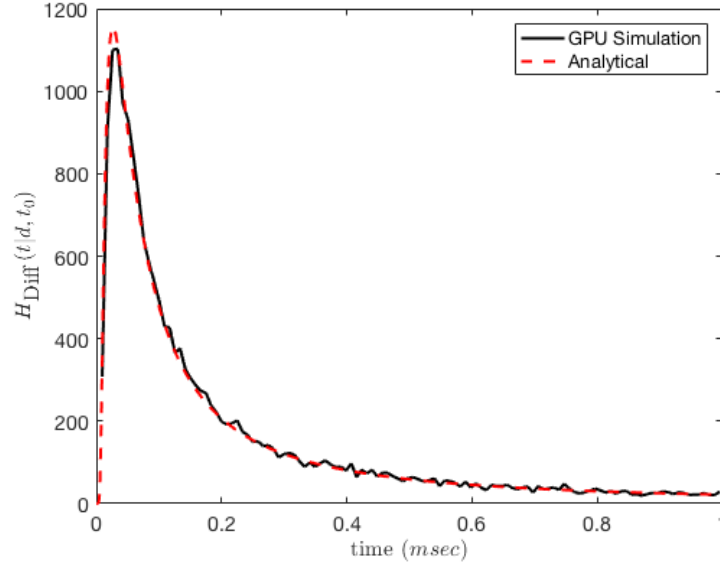


Figure 4.1: Hit probability for a single receiver, without drift, at  $d = 50\text{nm}$  from transmitter.

As can be seen in 4.1, the GPU simulation matches the analytical solution very well. Due to the time-step chosen,  $1\text{E-}7$ , the graph is missing the first leg. If a smaller time step was chosen, the missing leg would be smaller.

The next test used for validation looks at the probability a particle will reach a point in  $\mathbb{R}^1$ . This calculation is done in  $\mathbb{R}^1$  due to the lack of an analytical solution with drift in  $\mathbb{R}^3$ . The particle begins at a point  $\xi_0$ , at time  $t_0$  and will be recorded if it reaches a point,  $\xi_z$ . Each tracked particle is acted on by Brownian motion, as well as a laminar drift in the  $z$  direction. Therefore, with enough time each particle will reach the point  $\xi_z$ . This means that the hit probability is  $\Pr(m_k \text{ hit } \mathbf{R}) = 1$ , the same cannot be said for channels where  $\frac{a}{d} \ll 1$  in  $\mathbb{R}^3$ . The first-hitting time follows an inverse Gaussian distribution represented as

$$p_{\text{passage}}(t) = \frac{b}{\sqrt{4\pi D_B t^3}} \exp\left(\frac{-(vt - b^2)}{4D_B t}\right), \quad (4.3)$$

where  $b$  is the distance from the transmitter to the receiver, and  $v$  is the drift velocity, all in  $\mathbb{R}^1$ .



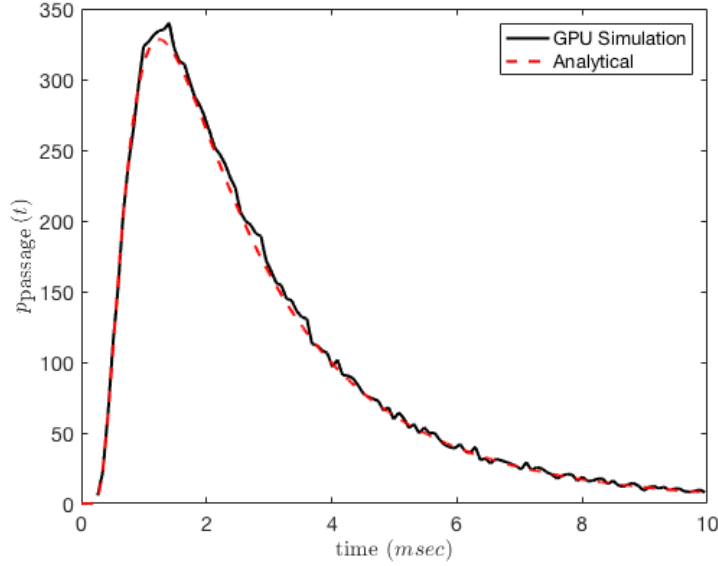


Figure 4.2: Hit probability for a single receiver, with drift, at  $b = 300\text{nm}$  from transmitter in  $\mathbb{R}^1$

The GPU output matches the analytical solution well. It is calculated using the first-hit time from 100,000 paths. This means that only the first point to pass the  $z$  threshold was utilized for this test.

With these two tests, it is clear that the movement of the particle via Brownian motion and Laminar drift behave according to proven algorithms. This gives a degree of confidence to any results used from this program.

## 4.2 Speed Comparison

The testing was done on a Nvidia GeForce GTX 1070 and the CPU used was an Intel I7 Kaby Lake, Quad-Core 4.2GHz processor. The GTX 1070 uses the GP104 chip referenced in, 2.1, however it only has 15/20 SMs enabled, vs the GTX 1080 that has the full 20 enabled. The CPU is run in a single core mode for the testing, it would likely increase the speed to some degree if the comparison algorithm were to be implemented using multiple cores. Three different tests were run. The first was used to as an eye test, to verify the output looked correct. It will be useful whenever a visual representation of the data is required. It can also be helpful when writing new new functions as a fast way to determine if it looks correct.

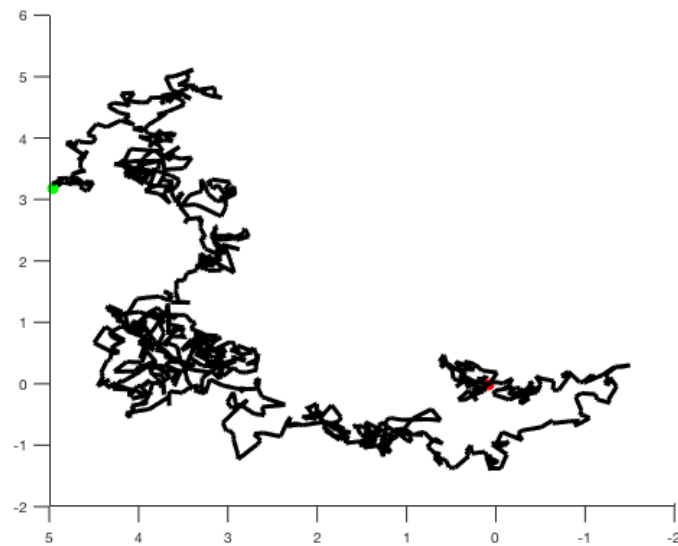


Figure 4.3: The path of a single particle without drift

In 4.4, it can be seen with a low number of paths calculated, the CPU is faster as expected. As the number of paths increase the GPU surpasses the CPU, the increases in speed on both processors appear to increase at a linear speed from that point. This limitation is due to the need to write at each step. The write requires a copy of the data back from GPU memory to local memory, then the CPU is used for to write the data to a CSV file. The data avoids being copied back to the GPU and is instead altered from its previous value.

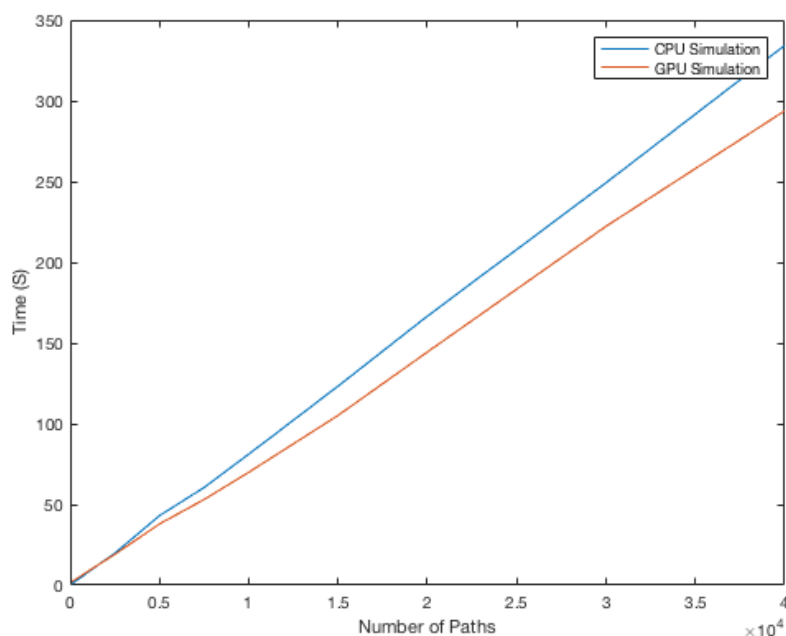


Figure 4.4: Speed comparison with each step recorded

The next tests that were performed compared the speed increases for the calculation of the first time found. These results were very interesting and some what disappointing for the purposes of this project. In 4.2, it can be seen that the GPU has a significant increase in speed for the first several thousand paths. However, about  $1\text{E}5$  paths, we see the saving in time change from increasing to decreasing. By the time  $1.7\text{E}5$ , the GPU begins to take longer. The other item of note, is that there is a very minimal difference in the calculation time of molecules with or without drift.

| Paths Simulated | CPU-No Drift | GPU-No Drift | CPU-Drift | GPU-Drift | Speed-up |
|-----------------|--------------|--------------|-----------|-----------|----------|
| 1,000           | 1.67s        | 2.25s        | 1.64s     | 2.27s     | .72x     |
| 5,000           | 8.24s        | 3.34s        | 8.26s     | 3.46s     | 2.39x    |
| 10,000          | 16.41s       | 4.55s        | 16.54s    | 4.81s     | 3.44x    |
| 50,000          | 87.6s        | 46.4s        | 82.66s    | 41.52s    | 1.99x    |
| 100,000         | 164.04s      | 109.833s     | 166.04s   | 111.98s   | 1.48x    |
| 200,000         | 349.8s       | 369.3s       | 334s      | 350s      | .95x     |

: First-hit CPU vs. GPU Speed Comparisons

Lastly, it was important to look at the speedup when compared to the number of paths run. 4.5 showed a very rapid increase for the first-hit time recordings, followed by a very quick drop off. The speed multiplier when recording all points was much more stable. It can be seen that around 1.5E4, the biggest speed up is found.

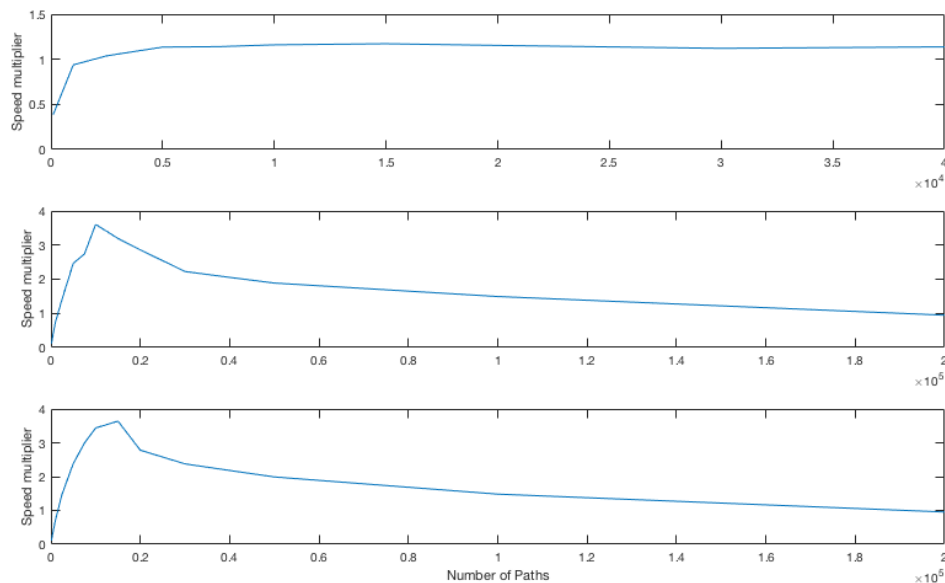


Figure 4.5: Speed multiplier per number of paths run

## 5 Discussion

The results found were interesting across all levels. The minimal level speed increase from CPU to GPU when recording all results is likely due to the data moving across memory, as well as the costs to write to the CSV file. The program was tested with all of the memory copies disabled and data writing removed, then compared to the different writes in order to better understand the speed differences seen. The program was run under three different scenarios. The first run was just the GPU, no memory was transferred back to host machine. The second run copied the memory to the host machine, but did nothing with it. The third copied memory back and wrote the first hit data to the file. The findings in 5 indicate that the way the GPU is called or the algorithm used in the GPU is likely the main limiting factor. The GPU is called in a loop from the CPU, in order to record each time step, as well as to enable future work that may look to enable the tracking of the collision of released molecules. If these requirements are ignored, the calculation of each path maybe instead

calculated in full by individual threads. This removes a significant amount of the communication back to the host machine and may result in better comparison speeds.

| Paths Simulated | GPU Only | Memory Transfer | Memory Transfer & Data Write |
|-----------------|----------|-----------------|------------------------------|
| 10000           | 3.8s     | 4.6s            | 5.7s                         |
| 50000           | 37s      | 40s             | 46s                          |
| 100000          | 104s     | 110s            | 121s                         |

: Reduction Speed Comparison

The original results saw much greater speedups, but in looking at the larger scope it was clear the full weight of the function was not being measured. In order to best compare the speed of the functions, the full implementation from start to finish was included in the speed calculations. This includes things such as memory transfer and data writing. This resulted in more meager looking speed ups. In the paper, [13] it finds that the average increase from the tested CPU to GPU was 2.5x. This is generally inline with our findings.

The other thing of interest was the slowdown seen by the GPU once the number of paths exceeded 1E5. This result is less intuitive to understand. I would like to implement the threads as discussed above and see if that same limitation is found. The block size, or fact that the GPU used is single-precision may have some effect on this. More research is needed. (I will hopefully have an answer this week)

The GPU used in this study was a single-precision GPU. Double precision GPUs do exist, but for this project they were deemed unnecessary. A single precision GPU will store 32 total bits, however one of those bits is a sign bit, and the next 8 are so-called exponent bits. This means that a significand precision of 24 bits can be expected. This equates to a maximum of  $3.4 \times 10^{38}$  and a minimum of  $1.2 \times 10^{-38}$ . These ranges encompass all calculations made, and expected to be made?

## 6 Future Work

There are many ways that this project can grow in the future. It can be customized to specific needs or tuned for increased speed as new technologies emerge. A few of the main things that maybe looked at in future studies are the implementation of Red Blood cells, Absorption vs reflection of molecules at the receiver and particle on particle reflection. Red blood cells are very influential

on the paths of the molecule as discussed throughout this paper. It would add accuracy to the simulation if the RBC could be added. It would also provide a new way of understanding the expected routes of molecules in the bloodstream. There are several challenges in doing so; the RBC can change in shape, they also continually rotate and the collisions will need to be handled for the molecules as well as the RBCs. The receivers simulated in this paper assume 100% absorption, but in reality, that is very unlikely. The receivers will be able to detect some of the particles, but depending on the design of the receiver, some of the particles may reflect or fail to be detected. Another possible future research opportunity would be particle on particle reflection. This would likely require multiple release points and a full data structure.

## **7 Conclusion**

This thesis was able to realize speed increases in the simulation of the particles for the purposes of the molecular communication. It also proved the validity of the results and opened the door for further research. There are many routes that maybe pursued in the future to continue this study.

## Vita

Author: Tobias Cain

Place of Birth: Nampa, Idaho

Undergraduate Schools Attended: Spokane Community College

Eastern Washington University

Degrees Awarded: Associate of Arts, 2011, Spokane Community College

Bachelor of Science in Electrical Engineering, 2015,

Eastern Washington University

## References

- [1] U. Rogers and M. S. Koh, “Exploring molecular distributed detection,” in *2015 9th IEEE International Conference on Nano/Molecular Medicine Engineering (NANOMED)*, Nov 2015, pp. 153–158.
- [2] B. Atakan, O. Akan, and S. Balasubramaniam, “Body area nanonetworks with molecular communications in nanomedicine,” vol. 50, no. 1, pp. 28–34, January 2012.
- [3] L. Felicetti, M. Femminella, G. Reali, P. Gresele, M. Malvestiti, and J. Daigle, “Modeling cd40-based molecular communications in blood vessels,” vol. 13, no. 3, pp. 230–243, Sept 2014.
- [4] J. Tan, A. Thomas, and Y. Liu, “Influence of red blood cells on nanoparticle targeted delivery in microcirculation,” *Soft Matter*, vol. 8, pp. 1934–1946, 2012.
- [5] U. Rogers, T. Cain, and M. Koh, “On spatial dependency in molecular distributed detection,” in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, March 2017, pp. 1063–1067.
- [6] T. Nakano, M. Moore, F. Wei, A. Vasilakos, and J. Shuai, “Molecular communication and networking: Opportunities and challenges,” vol. 11, no. 2, pp. 135–148, June 2012.
- [7] NVIDIA. (2016) NVIDIA GeForce GTX 1080 whitepaper. [Online]. Available: [https://international.download.nvidia.com/geforce-com/international/pdfs/GeForce\\_GTX\\_1080\\_Whitepaper\\_FINAL.pdf](https://international.download.nvidia.com/geforce-com/international/pdfs/GeForce_GTX_1080_Whitepaper_FINAL.pdf)
- [8] J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone, and J. C. Phillips, “Gpu computing,” *Proceedings of the IEEE*, vol. 96, no. 5, pp. 879–899, May 2008.
- [9] M. Januszewski and M. Kostur, “Accelerating numerical solution of stochastic differential equations with cuda,” *Computer Physics Communications*, vol. 181, no. 1, pp. 183 – 188, 2010. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0010465509002999>
- [10] U. Rogers and M. S. Koh, “Parallel molecular distributed detection with brownian motion,” vol. 15, no. 8, pp. 871–880, Dec 2016.



- [11] A. Pakrashi. (2013) Generating random numbers from normal distribution in c. [Online]. Available: <https://phoxis.org/2013/05/04/generating-random-numbers-from-normal-distribution-in-c/>
- [12] K. Schulten and I. Kosztin, “Lectures in theoretical biophysics,” *Champaign, IL, USA: University of Illinois at Urbana-Champaign*, vol. 117, 2000.
- [13] V. W. Lee, C. Kim, J. Chhugani, M. Deisher, D. Kim, A. D. Nguyen, N. Satish, M. Smelyanskiy, S. Chennupati, P. Hammarlund, R. Singhal, and P. Dubey, “Debunking the 100x gpu vs. cpu myth: an evaluation of throughput computing on cpu and gpu,” in *ISCA*, 2010.